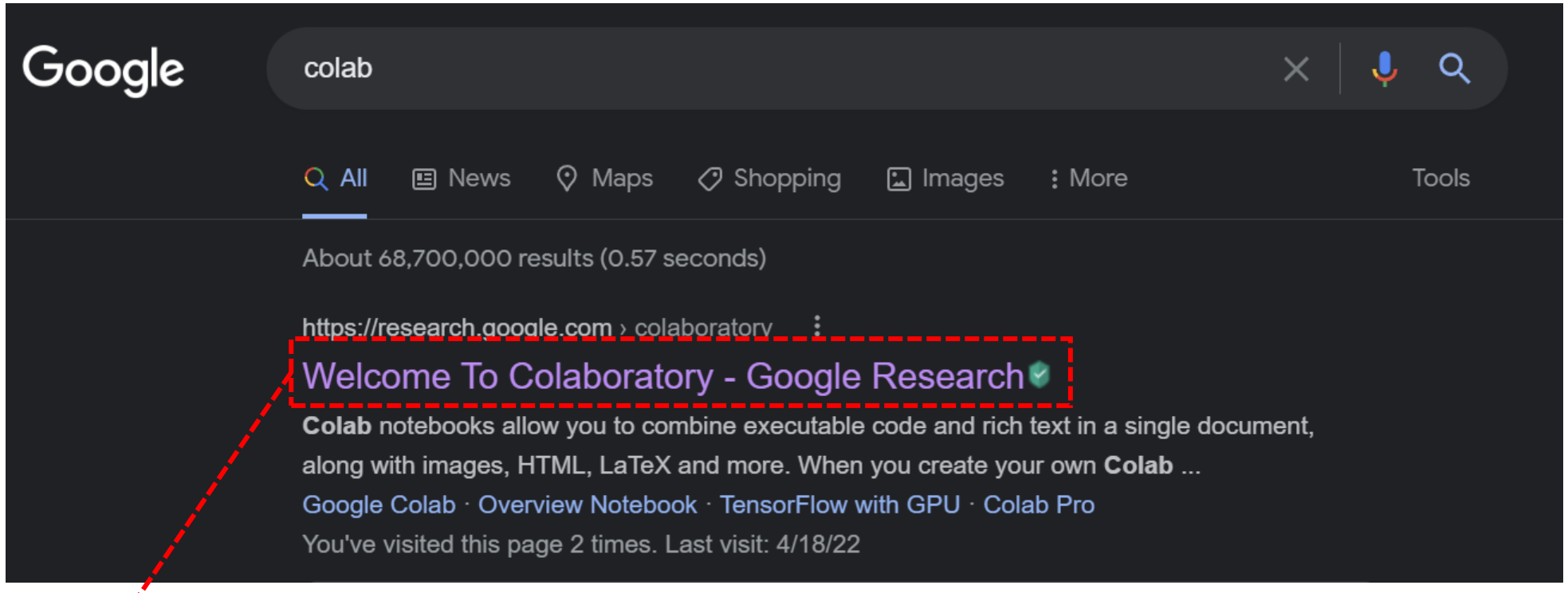
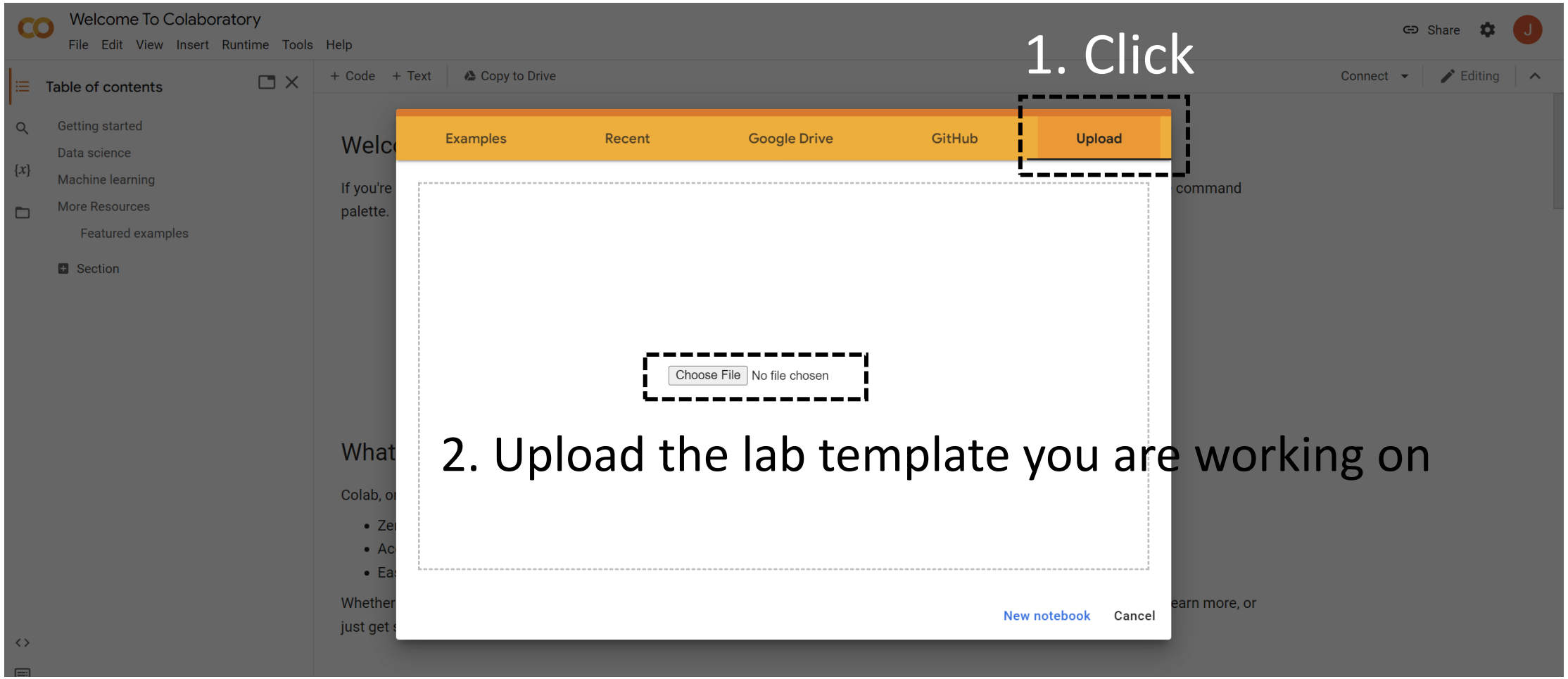
Step by step guide on Using Google Colab for Faster Training

University of Washington, Seattle

Spring 2022



Go to official Colab web page


Welcome To Colaboratory
File Edit View Insert Runtime Tools Help
Share
Table of contents
Getting started
Data science
Machine learning
More Resources
Featured examples
Section
+ Code + Text
Copy to Drive
Connect
Editing
1. Click
Examples
Recent
Google Drive
GitHub
Upload
Choose File No file chosen
2. Upload the lab template you are working on
New notebook
Cancel

Lab4_Template.ipynb

File Edit View Insert Runtime Tools Help Last saved at 12:06 PM

Comment Share

+ Code + Text

Connect Editing

Click

Lab 4 Report:

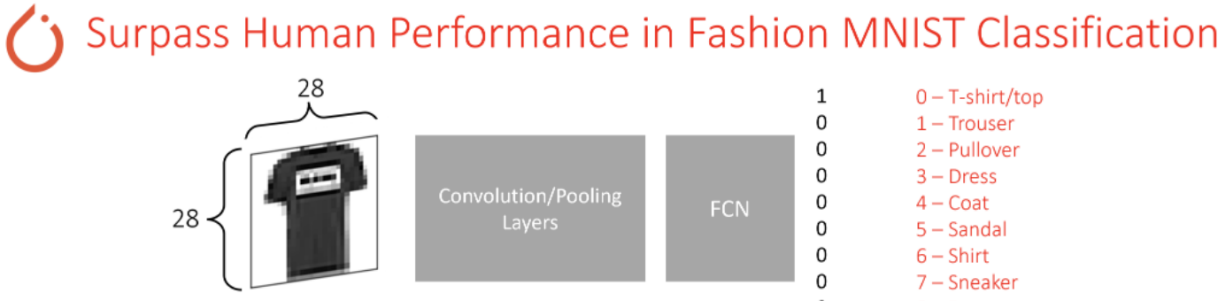
Surpass Human Performance in Fashion MNIST Classificaion

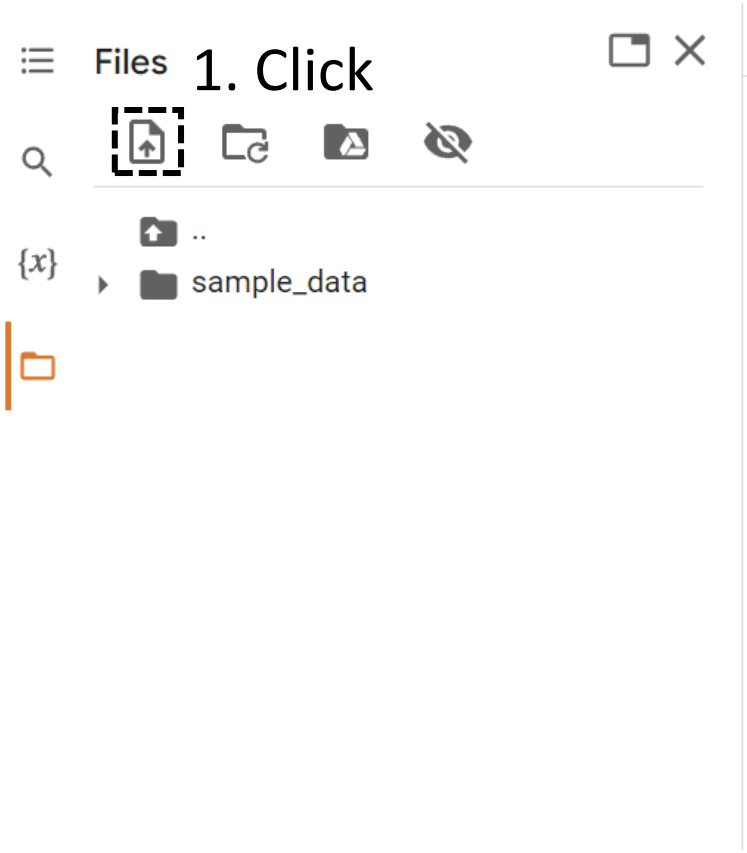
Name:

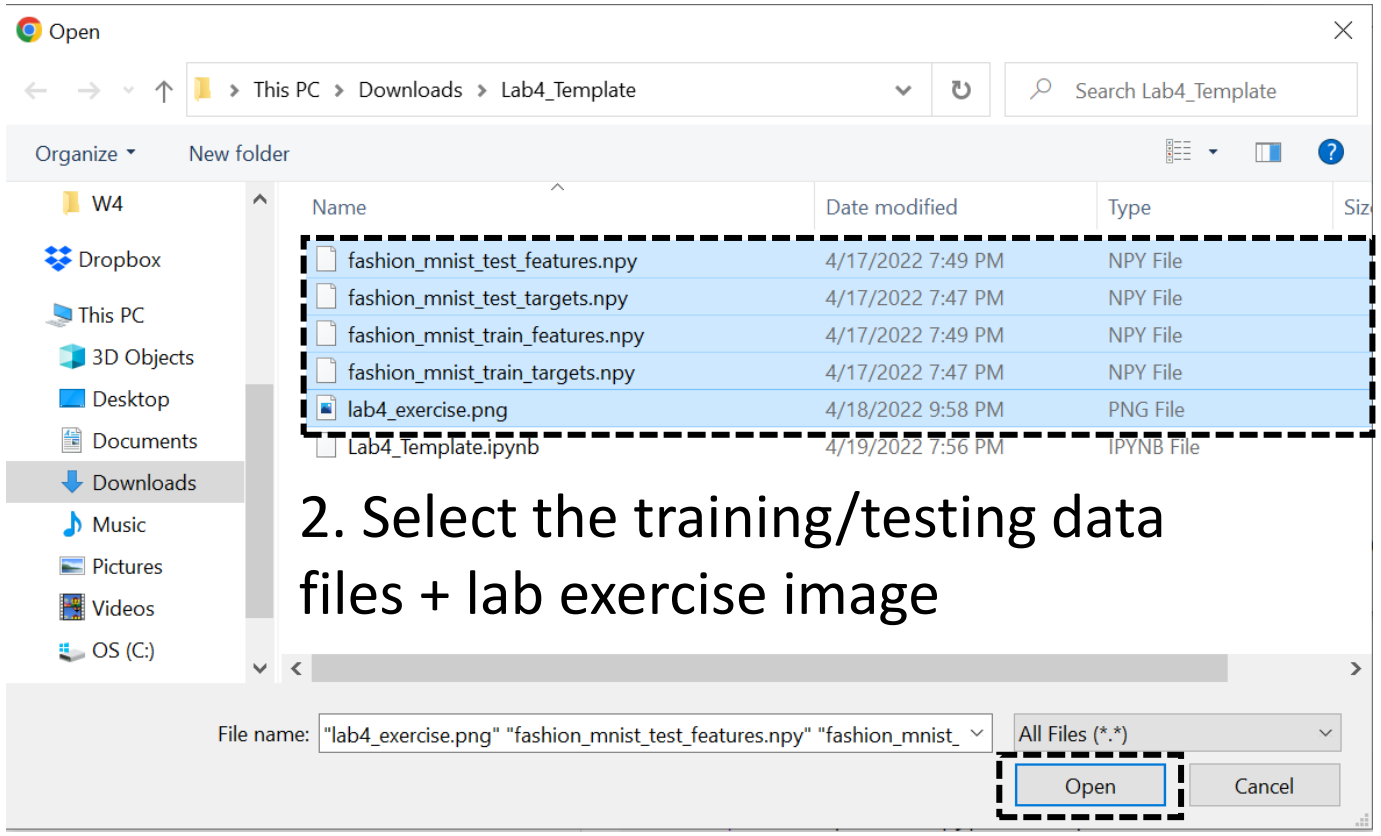
```
[ ] %matplotlib inline
    import matplotlib.pyplot as plt
    import torch
    import numpy as np
```

```
[ ] from IPython.display import Image # For displaying images in colab jupyter cell
```

```
[ ] Image('lab4_exercise.png', width = 1000)
```



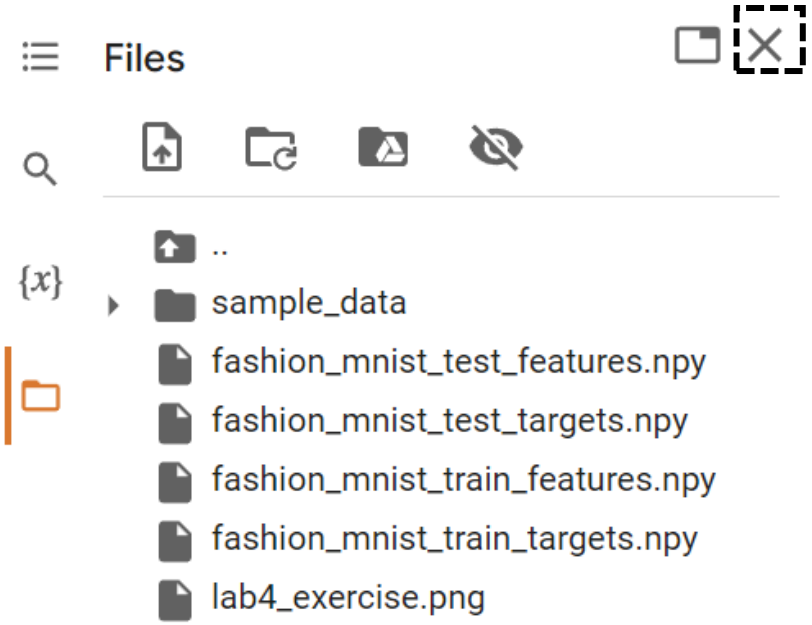




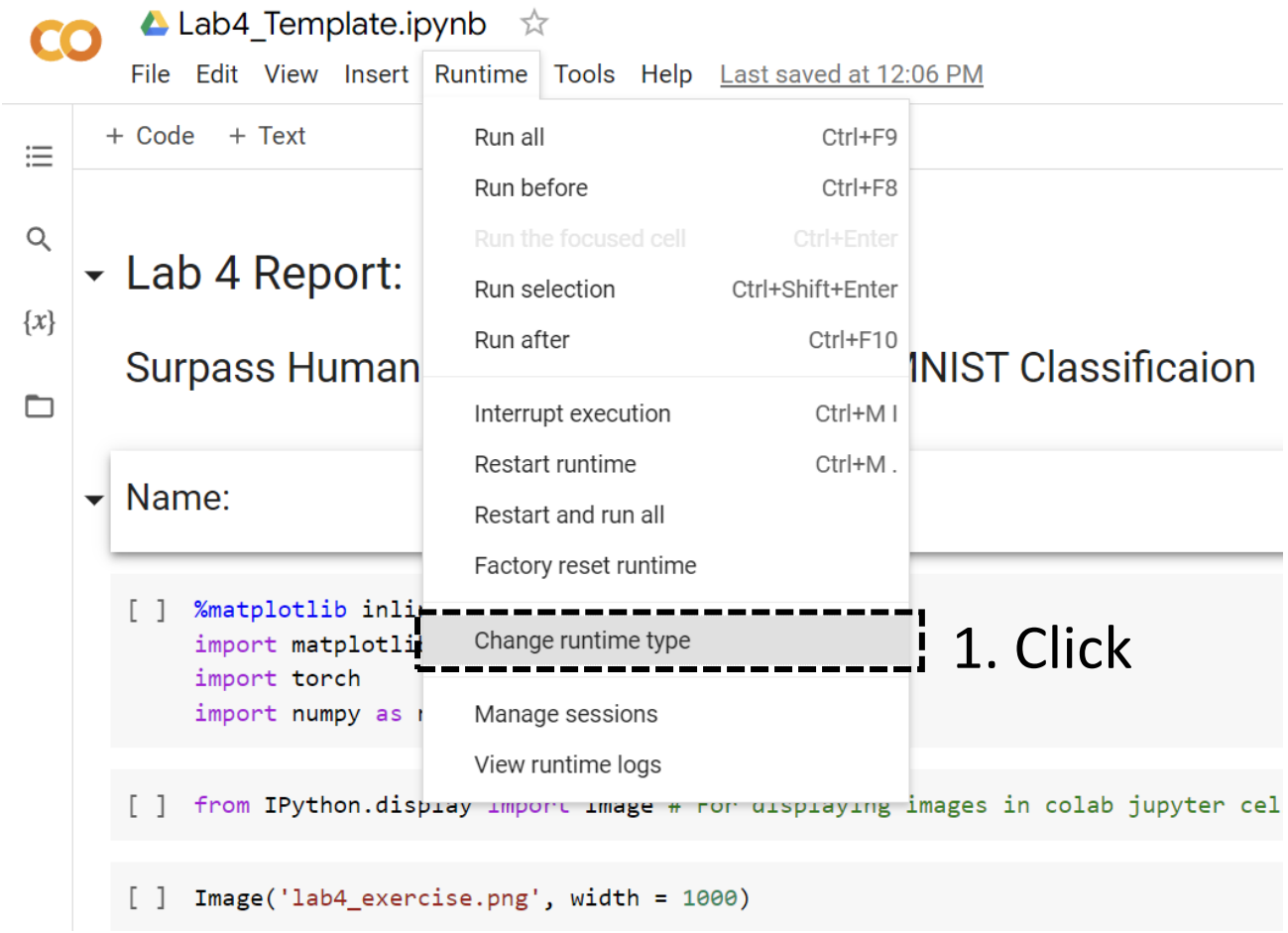
3. Upload

4. Click OK when prompted about uploaded files being deleted when runtime is recycled. You just need to reupload when they are deleted.

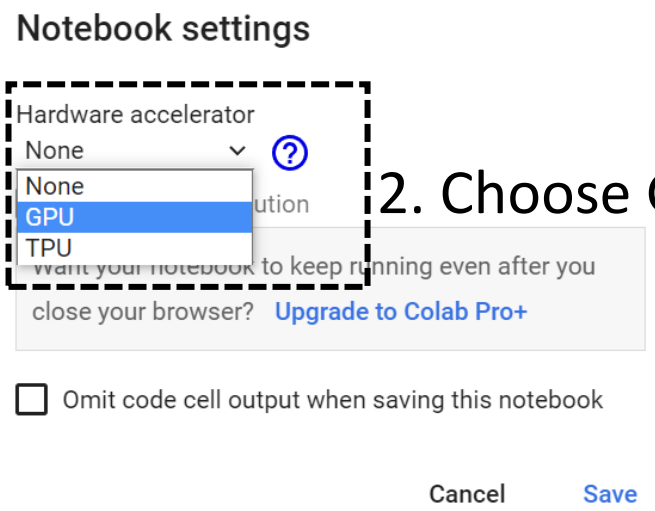
2. Click to close the file tab



1. You should now see all the necessary files for the lab are uploaded in Colab file system



1. Click



2. Choose GPU

3. If you exceeded the free usage limit for GPU, choosing None (CPU) will still give you decent performance

```
# CrossEntropyLoss as loss function and Adam as optimizer
# NOTE: CrossEntropyLoss automatically applies softmax to the output layer
# so no need to apply manual softmax to the output layer in forward function
loss_func = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)

# Print model specific. Add .cuda() to move your model to GPU
model.cuda()
```

```
train_inputs = torch.from_numpy(train_features).float().cuda()
train_targets = torch.from_numpy(train_targets).long().cuda()

validation_inputs = torch.from_numpy(validation_features).float().cuda()
validation_targets = torch.from_numpy(validation_targets).long().cuda()

testing_inputs = torch.from_numpy(test_features).float().cuda()
testing_targets = torch.from_numpy(test_targets).long().cuda()
```

If using GPU:

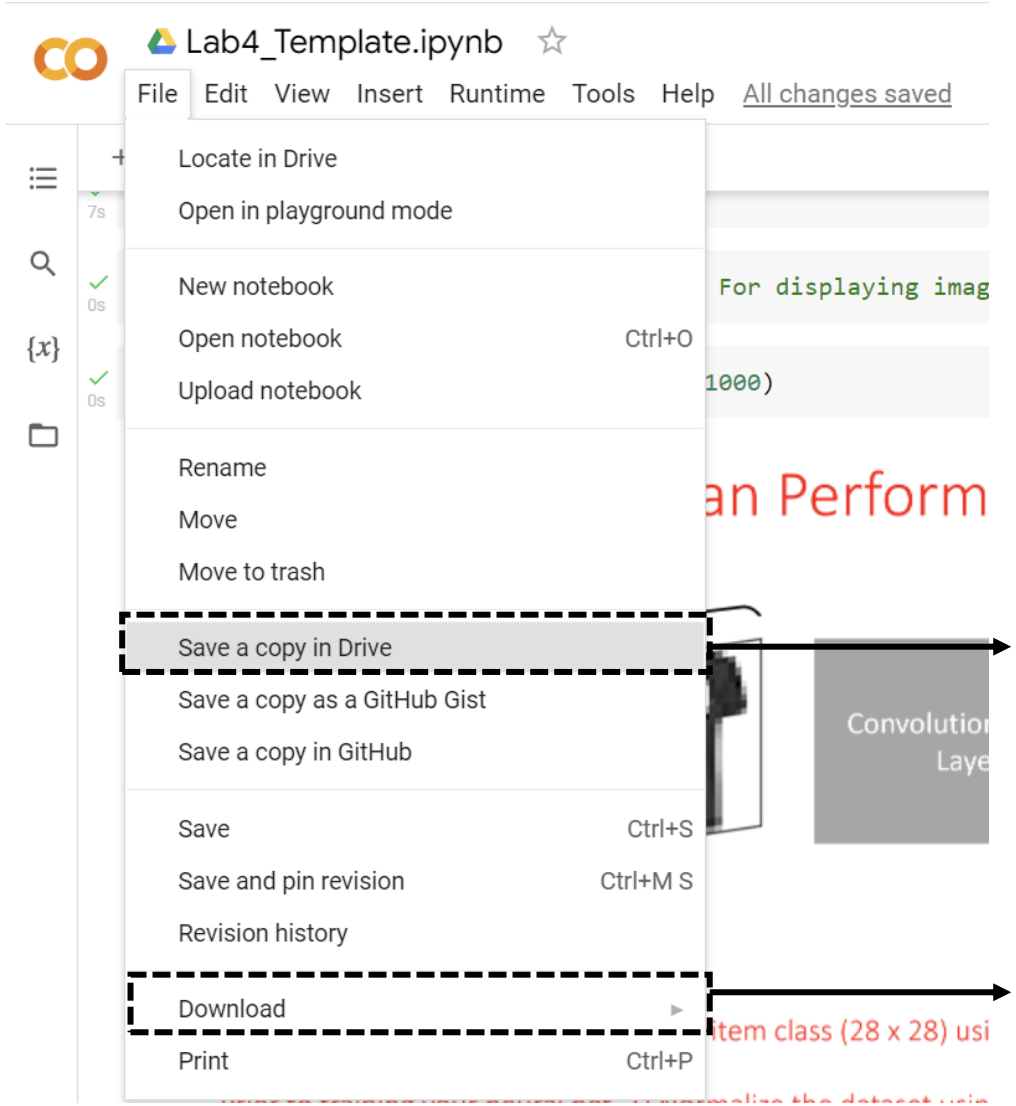
- Add .cuda() to your model object and torch tensors for your training/validation and testing sets.

If using CPU (i.e. None from Hardware accelerator):

- Remove .cuda() from both model and training/validation testing tensors

Refer to [Lab 2 slides]() to review transferring your tensors from GPU to CPU for post-training analysis

You are now ready to train your model in Colab. Feel free to DM me in Discord if you have any questions.
- Jimin



Save the copy of the notebook in your google drive

Download as .ipynb file